

Лабораторная работа №2

Анализ сложности алгоритмов. Шаблоны классов

Цель работы:

1. Научиться оценивать эффективность алгоритмов с помощью О-нотации (Big O)
2. Освоить классы сложности: $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$
3. Научиться проводить экспериментальный анализ производительности алгоритмов
4. Освоить создание шаблонов функций и классов для работы с различными типами данных
5. Получить навыки сравнения производительности различных реализаций одной задачи

Теоретическое обоснование

1. Что такое анализ сложности алгоритмов?

Анализ сложности - это математический способ оценки эффективности алгоритма в зависимости от размера входных данных.

Два вида сложности:

1. Временная сложность (Time Complexity) - сколько времени выполняется алгоритм
2. Пространственная сложность (Space Complexity) - сколько памяти использует алгоритм

2. О-нотация (Big O)

О-нотация - это математическая запись, описывающая асимптотическое поведение функции при стремлении аргумента к бесконечности.

Формальное определение:

Говорят, что $f(n) = O(g(n))$, если существуют положительные константы c и n_0 такие, что $0 \leq f(n) \leq c \times g(n)$ для всех $n \geq n_0$

Простым языком: О-нотация показывает, как быстро растёт время работы алгоритма при увеличении входных данных, отбрасывая константы и младшие члены.

Исходная формула	Упрощение	Сложность
$5n + 3$	n	$O(n)$
$2n^2 + 100n + 1000$	n^2	$O(n^2)$
$3n \log n + 4n$	$n \log n$	$O(n \log n)$
1000	1	$O(1)$

3. Основные классы сложности

Сложность	Название	Описание	Пример
$O(1)$	Константная	Время не зависит от размера данных	Доступ по индексу в массиве
$O(\log n)$	Логарифмическая	Увеличивается медленно, при удвоении данных время растёт на константу	Бинарный поиск
$O(n)$	Линейная	Время растёт пропорционально размеру данных	Линейный поиск, сумма элементов
$O(n \log n)$	Линейно-логарифмическая	Эффективные сортировки	Быстрая сортировка, слиянием
$O(n^2)$	Квадратичная	Время растёт квадратично	Пузырьковая сортировка
$O(2^n)$	Экспоненциальная	Время удваивается при добавлении одного элемента	Рекурсивное вычисление Фибоначчи
$O(n!)$	Факториальная	Самый медленный рост	Генерация всех перестановок

4. Сравнение классов сложности

n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(2^n)$
10	1	3	10	33	100	1 024
100	1	7	100	664	10 000	$\sim 1 \times 10^{30}$
1000	1	10	1 000	9 966	1 000 000	невозможно

n	O(1)	O(log n)	O(n)	O(n log n)	O(n ²)	O(2 ⁿ)
1 000 000	1	20	1 000 000	20 000 000	1×10 ¹²	НЕВОЗМОЖНО

5. Определение сложности по коду

```
// O(1) - константная
int getFirst(int arr[], int n) {
    return arr[0]; // одна операция, не зависит от n
}
```

```
// O(n) - линейная
int sum(int arr[], int n) {
    int total = 0;
    for (int i = 0; i < n; i++) { // n итераций
        total += arr[i];
    }
    return total;
}
```

```
// O(n2) - квадратичная (два вложенных цикла)
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}
```

```
// O(log n) - логарифмическая (деление пополам)
int binarySearch(int arr[], int n, int x) {
    int left = 0, right = n - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == x) return mid;
        if (arr[mid] < x) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

6. Шаблоны функций

Шаблоны позволяют писать код, работающий с разными типами данных.

```
// Шаблон функции для поиска максимума
template <typename T>
T max(T a, T b) {
    return (a > b) ? a : b;
}

// Шаблон функции для обмена значений
template <typename T>
void swap(T& a, T& b) {
    T temp = a;
    a = b;
    b = temp;
}

// Шаблон функции для линейного поиска
template <typename T>
int linearSearch(const T arr[], int n, const T& value) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == value) return i;
    }
    return -1;
}

// Использование
int main() {
    int a = 5, b = 10;
    swap(a, b);           // swap<int>

    double x = 3.14, y = 2.71;
    swap(x, y);           // swap<double>

    string s1 = "Hello", s2 = "World";
    swap(s1, s2);         // swap<string>
}
```

7. Шаблоны классов

```
// Шаблонный класс Pair
template <typename T1, typename T2>
class Pair {
```

```

private:
    T1 first;
    T2 second;

public:
    Pair(const T1& f, const T2& s) : first(f), second(s) {}

    T1 getFirst() const { return first; }
    T2 getSecond() const { return second; }

    void setFirst(const T1& f) { first = f; }
    void setSecond(const T2& s) { second = s; }
};

```

```

// Шаблонный класс Stack
template <typename T>
class Stack {
private:
    T* data;
    int topIndex;
    int capacity;

public:
    Stack(int size = 10) : capacity(size), topIndex(-1) {
        data = new T[capacity];
    }

    ~Stack() { delete[] data; }

    void push(const T& value) {
        if (topIndex < capacity - 1) {
            data[++topIndex] = value;
        }
    }

    T pop() {
        if (topIndex >= 0) {
            return data[topIndex--];
        }
        throw std::out_of_range("Stack is empty");
    }

    bool isEmpty() const { return topIndex == -1; }
};

```

8. Экспериментальный анализ производительности

```
#include <iostream>
#include <chrono>

using namespace std;
using namespace chrono;

// Функция для измерения времени выполнения
template <typename Func>
long long measureTime(Func func) {
    auto start = high_resolution_clock::now();
    func();
    auto end = high_resolution_clock::now();
    return duration_cast<microseconds>(end - start).count();
}

int main() {
    // Измерение времени линейного поиска
    const int N = 100000;
    int arr[N];
    for (int i = 0; i < N; i++) arr[i] = i;

    auto time = measureTime([&] () {
        for (int i = 0; i < 1000; i++) {
            linearSearch(arr, N, N - 1);
        }
    });

    cout << "Время выполнения: " << time << " мкс" << endl;
}
```

Примеры выполнения практической части

Пример 1. Определение сложности алгоритма (базовый уровень)

Задача: Определить сложность фрагментов кода и обосновать ответ.

```
#include <iostream>
using namespace std;

// Пример A: O(n)
int exampleA(int arr[], int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) { // n итераций
```

```

        sum += arr[i];           // O(1)
    }
    return sum;
}

```

// Пример B: $O(n^2)$

```

int exampleB(int arr[], int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {    // n итераций
        for (int j = 0; j < n; j++) { // n итераций для каждого i
            if (arr[i] == arr[j]) count++;
        }
    }
    return count;
}

```

// Пример C: $O(\log n)$

```

int exampleC(int n) {
    int count = 0;
    for (int i = 1; i < n; i *= 2) { // i удваивается:  $\log_2 n$  итераций
        count++;
    }
    return count;
}

```

// Пример D: $O(n \log n)$

```

int exampleD(int n) {
    int count = 0;
    for (int i = 0; i < n; i++) {    // n итераций
        for (int j = n; j > 0; j /= 2) { //  $\log_2 n$  итераций
            count++;
        }
    }
    return count;
}

```

// Пример E: $O(\sqrt{n})$

```

int exampleE(int n) {
    int count = 0;
    for (int i = 0; i * i < n; i++) { // i до  $\sqrt{n}$ 
        count++;
    }
    return count;
}

```

```

int main() {
    cout << "Анализ сложности алгоритмов:" << endl;
    cout << "exampleA: O(n) - линейный проход" << endl;
    cout << "exampleB: O(n²) - два вложенных цикла" << endl;
    cout << "exampleC: O(log n) - удвоение шага" << endl;
    cout << "exampleD: O(n log n) - комбинация линейного и
логарифмического" << endl;
    cout << "exampleE: O(√n) - условие i*i < n" << endl;
    return 0;
}

```

Пример 2. Сравнение производительности алгоритмов (средний уровень)

Задача: Реализовать линейный и бинарный поиск. Сравнить их производительность на массивах разного размера.

```

#include <iostream>
#include <vector>
#include <chrono>
#include <algorithm>

using namespace std;
using namespace chrono;

// Линейный поиск O(n)
template <typename T>
int linearSearch(const vector<T>& arr, const T& value) {
    for (int i = 0; i < arr.size(); i++) {
        if (arr[i] == value) return i;
    }
    return -1;
}

// Бинарный поиск O(log n)
template <typename T>
int binarySearch(const vector<T>& arr, const T& value) {
    int left = 0, right = arr.size() - 1;
    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (arr[mid] == value) return mid;
        if (arr[mid] < value) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

```

```

// Измерение времени выполнения
template <typename Func>
long long measureTime(Func func) {
    auto start = high_resolution_clock::now();
    func();
    auto end = high_resolution_clock::now();
    return duration_cast<microseconds>(end - start).count();
}

int main() {
    setlocale(LC_ALL, "ru");

    vector<int> sizes = {10000, 100000, 500000, 1000000};

    cout << "=== Сравнение линейного и бинарного поиска ===" << endl;
    cout << "Размер массива\tЛинейный (мкс)\tБинарный (мкс)" << endl;

    for (int n : sizes) {
        vector<int> arr(n);
        for (int i = 0; i < n; i++) arr[i] = i;

        int target = n - 1; // ищем последний элемент

        long long linearTime = measureTime([&](){
            linearSearch(arr, target);
        });

        long long binaryTime = measureTime([&](){
            binarySearch(arr, target);
        });

        cout << n << "\t\t" << linearTime << "\t\t" << binaryTime << endl;
    }

    return 0;
}

```

Ожидаемый вывод:

```

=== Сравнение линейного и бинарного поиска ===
Размер массива  Линейный (мкс)  Бинарный (мкс)
10000          45              1
100000         458             1
500000         2312            1
1000000        4678            2

```

Пример 3. Шаблонный класс Stack с анализом сложности (продвинутый уровень)

Задача: Реализовать шаблонный класс Stack с операциями push, pop, top. Проанализировать сложность каждой операции.

```
#include <iostream>
#include <stdexcept>

using namespace std;

template <typename T>
class Stack {
private:
    T* data;
    int topIndex;
    int capacity;

    void resize() {
        int newCapacity = capacity * 2;
        T* newData = new T[newCapacity];
        for (int i = 0; i <= topIndex; i++) {
            newData[i] = data[i];
        }
        delete[] data;
        data = newData;
        capacity = newCapacity;
        cout << "Расширение стека до " << capacity << " элементов" << endl;
    }

public:
    Stack(int initialCapacity = 10)
        : capacity(initialCapacity), topIndex(-1) {
        data = new T[capacity];
        cout << "Создан стек ёмкостью " << capacity << endl;
    }

    ~Stack() {
        delete[] data;
        cout << "Стек уничтожен" << endl;
    }

    // O(1) в среднем, O(n) при расширении (амортизированно O(1))
    void push(const T& value) {
        if (topIndex >= capacity - 1) {
            resize();
        }
    }
};
```

```

    }
    data[++topIndex] = value;
}

// O(1)
T pop() {
    if (isEmpty()) {
        throw out_of_range("Стек пуст");
    }
    return data[topIndex--];
}

// O(1)
T top() const {
    if (isEmpty()) {
        throw out_of_range("Стек пуст");
    }
    return data[topIndex];
}

// O(1)
bool isEmpty() const {
    return topIndex == -1;
}

// O(1)
int size() const {
    return topIndex + 1;
}

// O(n) - для демонстрации
void print() const {
    cout << "[";
    for (int i = 0; i <= topIndex; i++) {
        cout << data[i];
        if (i < topIndex) cout << ", ";
    }
    cout << "]" << endl;
}

};

int main() {
    setlocale(LC_ALL, "ru");

    cout << "=== Анализ сложности операций стека ===" << endl;

```

```

    cout << "push: O(1) амортизированно (иногда O(n) при расширении)"
<< endl;
    cout << "pop: O(1)" << endl;
    cout << "top: O(1)" << endl;
    cout << "isEmpty: O(1)" << endl;
    cout << "size: O(1)" << endl;
    cout << "print: O(n)" << endl << endl;

    Stack<int> s(4);

    cout << "Добавление элементов:" << endl;
    for (int i = 1; i <= 10; i++) {
        s.push(i);
        cout << "push(" << i << "), размер: " << s.size() << endl;
    }

    cout << "\nСодержимое стека: ";
    s.print();

    cout << "\nУдаление элементов:" << endl;
    while (!s.isEmpty()) {
        cout << "pop() = " << s.pop() << ", размер: " << s.size() << endl;
    }

    return 0;
}

```

Индивидуальные задания

Базовый уровень (15 вариантов)

Общее задание: Для фрагментов кода (выдаётся преподавателем) определить сложность каждого фрагмента и обосновать ответ. Реализовать шаблонную функцию, указанную в варианте.

№ вар.	Шаблонная функция	Назначение
1	$T \max(T a, T b)$	Возвращает максимальное из двух значений
2	$T \min(T a, T b)$	Возвращает минимальное из двух значений

№ вар.	Шаблонная функция	Назначение
3	T abs(T x)	Возвращает абсолютное значение (для числовых типов)
4	void swap(T& a, T& b)	Обменивает значения двух переменных
5	T sum(T arr[], int n)	Возвращает сумму элементов массива
6	T average(T arr[], int n)	Возвращает среднее арифметическое
7	T findMax(T arr[], int n)	Возвращает максимальный элемент
8	T findMin(T arr[], int n)	Возвращает минимальный элемент
9	int linearSearch(T arr[], int n, T key)	Линейный поиск элемента
10	bool isEqual(T a, T b)	Сравнивает два значения на равенство
11	T product(T arr[], int n)	Возвращает произведение элементов
12	int countIf(T arr[], int n, T value)	Подсчитывает количество вхождений
13	void fill(T arr[], int n, T value)	Заполняет массив значением
14	T getOrDefault(T arr[], int n, int index, T defaultValue)	Возвращает элемент или значение по умолчанию
15	bool allEqual(T arr[], int n)	Проверяет, все ли элементы равны

Средний уровень (15 вариантов)

Общее задание: Реализовать шаблонный класс, указанный в варианте. Для каждой операции указать сложность. Провести экспериментальное сравнение производительности двух реализаций.

№ вар.	Класс	Операции	Сравнение
1	Pair<T1, T2>	Конструктор, getFirst, getSecond, setFirst, setSecond	—
2	Triple<T1, T2, T3>	Конструктор, геттеры, сеттеры	—
3	ArrayWrapper<T>	Конструктор, get, set, getSize	Линейный vs бинарный поиск
4	Counter<T>	add, getCount, reset, getMostFrequent	—
5	Range<T>	Конструктор, contains, getStart, getEnd	—
6	Nullable<T>	Конструктор, hasValue, getValue, reset	—
7	CircularBuffer<T>	push, pop, isEmpty, isFull	—
8	Accumulator<T>	add, getSum, getCount, getAverage	—
9	Filter<T>	Конструктор, apply, addCondition	Линейный vs бинарный поиск
10	Mapper<Tin, Tout>	Конструктор, apply, addMapping	—
11	Cache<T>	get, put, contains, clear	—
12	History<T>	push, undo, redo, clear	—
13	Observable<T>	setValue, getValue, addObserver	—

№ вар.	Класс	Операции	Сравнение
14	Lazy<T>	get, isInitialized, reset	—
15	Pool<T>	acquire, release, getAvailable	—

Продвинутый уровень (15 вариантов)

Общее задание: Реализовать шаблонный контейнер с заданной функциональностью. Провести анализ сложности каждой операции. Экспериментально сравнить производительность с аналогом из STL.

№ вар.	Контейнер	Требования
1	Vector<T>	push_back, pop_back, operator[], size, capacity, reserve
2	List<T>	push_front, push_back, pop_front, pop_back, size, insert, erase
3	Stack<T>	push, pop, top, isEmpty, size
4	Queue<T>	enqueue, dequeue, front, isEmpty, size
5	Deque<T>	push_front, push_back, pop_front, pop_back, operator[]
6	PriorityQueue<T>	push, pop, top, isEmpty, size (на основе кучи)
7	Set<T>	insert, contains, remove, size (на основе BST)
8	HashMap<K, V>	put, get, containsKey, remove, size
9	Array<T>	operator[], fill, sort, find (с динамическим расширением)
10	SortedList<T>	insert, contains, remove, getByIndex (сохраняет порядок)
11	RingBuffer<T>	push, pop, operator[], isEmpty, isFull (циклический буфер)

№ вар.	Контейнер	Требования
12	SparseArray<T>	set, get, remove, size (разрежённый массив)
13	StaticVector<T, N>	push_back, pop_back, operator[], size (фиксированная ёмкость)
14	FixedQueue<T, N>	enqueue, dequeue, front, isEmpty, isFull
15	BitSet	set, clear, test, count, operators &, , ^

Контрольные вопросы

1. Что такое O-нотация (Big O) и зачем она нужна?
2. Какая сложность у доступа к элементу массива по индексу? Почему?
3. Какая сложность у бинарного поиска? Почему?
4. Что означает сложность $O(n \log n)$? Приведите пример алгоритма.
5. Что такое амортизированный анализ? Приведите пример.
6. Какая сложность у двух вложенных циклов по n итераций каждый?
7. Какая сложность у рекурсивного вычисления чисел Фибоначчи?
8. Что такое шаблоны функций? Как их объявить и использовать?
9. Что такое шаблоны классов? Как их объявить и использовать?
10. В чём преимущество шаблонов перед перегрузкой функций?
11. Как указать, что шаблонный параметр - тип?
12. Можно ли создать шаблон с несколькими параметрами типов?
13. Как провести экспериментальное измерение времени выполнения?
14. Почему при измерении времени нужно выполнять операцию многократно?
15. Что важнее: сложность алгоритма или константа?

Требования к отчёту

Титульный лист с названием работы, ФИО студента, номером группы, вариантом

Цель работы (формулируется самостоятельно на основе предложенной)

Теоретическая часть (основные понятия: O-нотация, классы сложности, шаблоны)

Постановка задачи (текст задания согласно варианту)

Листинг программы с комментариями

Результаты работы (скриншоты вывода программы, результаты измерений)

Анализ сложности (для каждой реализованной операции указать сложность)

Ответы на контрольные вопросы (письменно, развёрнуто)

Выводы (что нового узнали, какие навыки приобрели)

Ссылка на Git-репозиторий с кодом программы

Критерии оценивания

Базовый уровень (максимум 60 баллов)

Критерий	Баллы
Правильное определение сложности фрагментов кода	20
Реализация шаблонной функции согласно варианту	15
Демонстрация работы с разными типами данных	10
Код отформатирован, есть комментарии	10
Есть ответы на контрольные вопросы	5

Средний уровень (максимум 80 баллов)

Критерий	Баллы
Все требования базового уровня	40
Реализация шаблонного класса согласно варианту	15
Указание сложности для каждой операции класса	10
Экспериментальное сравнение двух реализаций	15

Продвинутый уровень (максимум 100 баллов)

Критерий	Баллы
Все требования среднего уровня	60
Реализация шаблонного контейнера с полной функциональностью	15
Анализ сложности всех операций	10
Сравнение с STL-аналогом (таблица, график)	10

Критерий	Баллы
Наличие ссылки на Git-репозиторий	5

Штрафные баллы

Нарушение	Штраф
Неправильное определение сложности	-10
Ошибки в шаблонном синтаксисе	-10
Отсутствуют комментарии в коде	-5
Нет ответов на контрольные вопросы	-10
Нет ссылки на Git-репозиторий	-10
Программа не компилируется	работа принимается не

Итоговая шкала

Оценка (традиционная)	Баллы
Отлично	90–100
Хорошо	75–89
Удовлетворительно	60–74
Неудовлетворительно	менее 60